Dynamic Programming

Reading companion. References below to **M&F** mean Mario J. Miranda and Paul L. Fackler (2002), *Applied Computational Economics and Finance*, MIT Press, Chapters 7 and 8 (course references). Page numbers are the book's printed page numbers, not the PDF viewer's page count. Chapter 7 develops discrete-state models and algorithms; Chapter 8 develops continuous-state theory. Our fishery discussion draws on Chapter 8 for the economic reasoning and Chapter 7 for the discrete numerical solution.

Translating notation. M&F use f for the reward (our π), g for the state transition (our f), and δ for the discount factor (our β). Their shadow price $\lambda(s)$ is $V'(s)$. Their finite-horizon convention has decisions at $1, \ldots, T$ followed by terminal value V_{T+1}; our general finite-horizon formulation uses decisions at $0, \ldots, T-1$ followed by V_T. Our examples simplify or adapt the book's models, so their numerical answers need not match ours.

1 Motivation: Why Dynamics Matter

- Many economic problems involve *decisions over time*.
- In static optimization, we choose a single vector x to maximize $f(x)$ once.
- In dynamic optimization, we choose an entire sequence $\{x_t\}_{t=0}^{T}$ to maximize $\sum_{t=0}^{T} \beta^t \pi(s_t, x_t)$

The key feature of a dynamic problem is **intertemporal dependence** - today's decision affects tomorrow's feasible set.

Consider a groundwater extraction problem:

$$\max_{q_1, q_2} \ \pi_1(q_1) + \beta \pi_2(q_2)$$

subject to

$$s_{t+1} = s_t - q_t, \quad 0 \leq q_t \leq s_t, \quad t = 1, 2.$$

Let's assume it will not be optimal to leave any resource in the ground at the end, so $s_3 = 0$ and $q_2 = s_2 = s_1 - q_1$. Substituting q_2 into the objective gives

$$\max_{q_1} \ \pi_1(q_1) + \beta \pi_2(s_1 - q_1).$$

The first-order conditions give

$$\frac{\partial \left[\pi_1(q_1) + \beta \pi_2(s_1 - q_1)\right]}{\partial q_1} = \pi_1'(q_1) - \beta \pi_2'(s_1 - q_1) = 0,$$

which equates the **marginal benefit of current extraction** to the **discounted marginal benefit of future extraction**.

Examples

- Resource extraction (trade-off between current and future use)
  - Suppose a manager extracts a nonrenewable resource such as groundwater or oil.
  - Each period, extraction reduces the remaining stock.
  - The manager faces a trade-off between the *immediate profit* from extracting today and the *future value* of the remaining stock.
  - Dynamic optimization determines the optimal extraction path balancing current and future gains — or the *shadow value* of the resource.
- Investment and capital accumulation
  - Firms decide how much to invest in physical capital (machinery, buildings) or human capital (training, education).
  - Investment today increases the productive capacity tomorrow, affecting future profits.
  - The firm must weigh the *cost of investment* against the *expected future returns*, leading to an optimal investment strategy over time.
- Crop rotations or livestock feeding
  - Farmers choose how to allocate land and resources over multiple periods.
  - Planting a crop today may deplete soil nutrients, affecting yields in subsequent seasons.
  - Similarly, feeding strategies for livestock impact their growth and productivity over time.
  - Dynamic programming helps determine optimal crop rotations or feeding schedules that maximize long-term farm profitability.
- Forest rotation and timber harvesting
  - Forest managers decide when to harvest trees to maximize the value of timber while ensuring sustainability.
  - Harvesting too early may yield lower quantities of wood, while waiting too long can lead to overgrowth where the marginal biomass growth does not justify the cost.
  - Dynamic optimization helps find the optimal rotation period that balances immediate revenue with future discounted value.
- Pollution control and climate policy
  - Governments and firms must decide how much to invest in pollution abatement technologies or carbon reduction strategies.
  - Reducing emissions today can mitigate future environmental damage and associated costs.
  - Dynamic programming helps evaluate the trade-offs between current abatement costs and long-term benefits, guiding optimal policy decisions over time.

1.1 Backward Induction

Further reading: M&F §7.1, pp. 156–157, explains the terminal condition and recursive logic; §7.3.1, pp. 164–165, gives the backward-recursion algorithm.

- In a *finite-horizon sequential decision problem*, one can solve by starting at the last period and then working backward.
- At the final period, the decision is trivial (or known) because nothing is left to optimize afterward.
- Then one moves to the second-to-last period: given that the future is solved optimally, choose your action now to maximize current payoff plus continuation value.
- This process continues backward until reaching the initial period.
- Backward induction is prefered over brute force enumeration because it simplifies potentially complex problems. Starting from the beginning, the number of paths can explode combinatorially, making it infeasible to evaluate all possible sequences of decisions.
- In a finite-horizon problem, backward induction works because there is a last period: after that, no decisions remain. We can anchor the recursion at T and move backward.
- In an infinite-horizon problem, there is no terminal period. We need a different anchor. What we look for is a stationary path (or policy) that is internally consistent over time: i.e. at every date, given the current state, the decision you make will also be optimal for the continuation. In other words, you want a path or policy such that there is no incentive to deviate at any point — because the same 'value' structure recurs over time. That is what gives the stability and time-independence of the optimal policy in many infinite-horizon problems.

1.2 Vocabulary and the components of dynamic programming problem

Further reading: M&F §7.1, pp. 155–157, defines discrete-state model elements. Compare §8.1, pp. 190–191, for continuous states and the distinction between continuous and discrete choices.

- **State variable** s_t: summarizes all relevant information at time t needed to make decisions and predict future states. Examples: resource stock, capital stock, productivity level.
- **Control variable** x_t: the decision made at time t. Examples: extraction quantity, investment amount, input usage.
- **Transition function** $s_{t+1} = f(s_t, x_t, \varepsilon_{t+1})$: describes how the state evolves from t to $t+1$ given current state, action, and random shocks. Also called the **law of motion** for the state variable.
- **Reward function** $\pi(s_t, x_t)$: the immediate payoff from taking action x_t in state s_t. Examples: profit, utility, yield.
- **Policy** $\{x_t(s_t)\}$: a rule that specifies the action to take in each state at each time.
- **Value function** $V_t(s_t)$: the maximum expected discounted sum of future rewards starting from state s_t at time t.
- **Discount factor** $\beta \in (0, 1)$: reflects the preference for current rewards over future rewards. It reflects time preference or impatience: how much less future utility / profit is worth relative to now. In growth / resource models, discounting ensures that long-run payoffs are "worth less" the farther they are in the future, which prevents over-emphasis on infinitely deferred rewards.
  - It also enables contraction mappings in infinite-horizon problems.
  - We often assume a constant discount factor for simplicity, but in reality, discounting may vary over time or depend on circumstances.
- **Salvage value** $V_T(s_T)$: the value of remaining resources or capital at the end of the planning horizon. In finite-horizon problems, this can be zero or some terminal value function.

1.3 A Taxonomy

Deterministic: The next state is completely determined by the current state and decision.

State space	Discrete time	Continuous time
Discrete	**Finite-state DP** Shortest path; knapsack; equipment replacement with known deterioration	**Deterministic switching / hybrid control** Scheduled machine switching; deterministic hybrid systems
Continuous	**Continuous-state DP** Capital accumulation; reservoir management with known inflows	**Deterministic optimal control** Ramsey growth; renewable resource harvesting

Stochastic: The next state depends on random shocks governed by a probability distribution or stochastic process.

State space	Discrete time	Continuous time
Discrete	**Markov Decision Process (MDP)** Stochastic inventory management; equipment replacement with random failures	**Continuous-Time MDP (CTMDP)** Queueing systems; equipment failure and repair
Continuous	**Stochastic continuous-state DP** Reservoir management with uncertain inflows; stochastic capital accumulation	**Stochastic optimal control** Stochastic fisheries; portfolio optimization; resource extraction with stochastic prices

2 Discrete time and state space

- Dynamic optimization problems describe sequential decision-making over time, where future states depend on current actions.
- In discrete time and discrete state settings, time proceeds in periods $t = 0, 1, \ldots, T$, and the system occupies one of a finite set of states $s_t \in S$ at each time t.
- The decision variable is denoted by $x_t \in X(s_t)$, representing the feasible choices when the system is in state s_t.

2.1 General Problem

- The general **finite-horizon** problem is to choose a sequence $\{x_t\}_{t=0}^{T-1}$ to maximize the expected discounted sum of single-period payoffs:

$$\max_{\{x_t\}_{t=0}^{T-1}} \sum_{t=0}^{T-1} \beta^t \pi(s_t, x_t) + \beta^T V_T(s_T)$$

where:

- $\beta \in (0, 1)$ is the discount factor,
- $\pi(s_t, x_t)$ is the single-period reward (or profit),
- $V_T(s_T)$ is the *terminal value function* (salvage value) at the end of the horizon.

The **transition dynamics** are deterministic:

$$s_{t+1} = f(s_t, x_t),$$

with an initial state s_0 given.

Does our simple 2 period problem fit into this framework?

2.2 Bellman Equation

Further reading: M&F §7.1, pp. 156–157, develops the principle of optimality, finite-horizon Bellman equation, and terminal values. The book includes uncertainty; our deterministic transition is the special case in which the next state is known.

- The recursive formulation expresses the problem in terms of a **value function** $V_t(s_t)$ that gives the maximal attainable value starting from state s_t at time t:

$$V_t(s_t) = \max_{x_t \in X(s_t)} \Big\{ \pi(s_t, x_t) + \beta\, V_{t+1}(f(s_t, x_t)) \Big\}.$$

- The **terminal condition** closes the recursion. Let $\pi_T(s_T)$ denote the terminal payoff, including any final-date choice already optimized:

$$V_T(s_T) = \pi_T(s_T)$$

- Imagine we arrive at date t with state s_t. We choose the action x_t, not the current state.
- Todays decision determines both today's payoff and tomorrow's state.
- All future decisions are summarized by the value function $V_{t+1}(s_{t+1})$.
- The largest attainable payoff is the value function $V_t(s)$; the action that achieves it is the policy $x_t^*(s)$.

The Bellman equation expresses the **principle of optimality**:

The value of being in state s_t is the maximum over current actions of the immediate payoff plus the discounted value of the next state.

2.2.1 Example: Extract or Wait?

Suppose we have one unit of a resource, $s_t = 1$. We can extract it today for a payoff of \$8, or wait and keep it for tomorrow. Tomorrow is the final decision date, and its higher price gives a harvest payoff of \$10. Thus $\pi_t(s_t, 1) = 8$ and $\pi_{t+1}(s_{t+1}, 1) = 10$ when one unit is available; waiting earns zero before the terminal date, and any resource left after tomorrow has zero salvage value.

With a discount factor of $\beta = 0.9$, compare the two actions:

Today's action	Payoff now	Discounted continuation value	Total
Extract	8	$0.9(0) = 0$	8
Wait	0	$0.9(10) = 9$	9

The Bellman equation makes this comparison:

$$V_t(1) = \max\left\{\underbrace{8 + 0.9(0)}_{\text{extract}}, \underbrace{0 + 0.9(10)}_{\text{wait}}\right\} = 9.$$

- The **value** is $V_t(1) = 9$: the best attainable discounted payoff from this state.
- The **policy** is $x_t^*(1) = 0$: wait and preserve the resource.

What current extraction payoff would make us indifferent between extracting and waiting?

We would be indifferent at \$9, the discounted value of keeping the resource for tomorrow.

2.2.2 Mining problem

Further reading: M&F §7.2.1, pp. 157–158, sets up mine management; §7.6.1, pp. 172–175, works through a numerical implementation and plots. Our three-period example uses a simpler payoff and smaller stock grid.

Consider a mine management problem, where:

- state: remaining ore s_t, and initial stock $s_0 = 4$,
- control: extraction $x_t \in \{0, 1, \dots, s_t\}$,
- transition: $s_{t+1} = s_t - x_t$,
- reward: $\pi(x_t) = 8x_t - x_t^2$, which depends only on extraction,
- terminal condition: $V_T(s_T) = 0$
- horizon: $T = 3$,
- discount factor: $\beta = 0.9$.

Set up the problem/Bellman equation and solve via backward induction.

We solve by **backward induction**:

$$V_t(s) = \max_{x \in \{0,\dots,s\}} \{\pi(x) + \beta V_{t+1}(s - x)\}, \quad V_3(s) \equiv 0.$$

2.2.3 Step 1: Final decision period $t = 2$

Here $V_3(\cdot) = 0$, so extract to maximize $\pi(x)$ subject to $x \le s$:

- $V_2(0) = 0$

- $V_2(1) = \pi(1) = 7$
- $V_2(2) = \pi(2) = 12$
- $V_2(3) = \pi(3) = 15$
- $V_2(4) = \pi(4) = 16$

Optimal action at $t = 2$ is $x_2^*(s) = s$ (extract all that remains).

2.2.4 Step 2: Period $t = 1$

Compute $V_1(s) = \max_{x \le s}\{\pi(x) + \beta V_2(s - x)\}$:

- $s = 0$: only $x = 0 \Rightarrow V_1(0) = 0$
- $s = 1$:
  - $x = 0$: $0 + 0.9 \cdot V_2(1) = 6.3$
  - $x = 1$: $7 + 0.9 \cdot 0 = 7$
    $\Rightarrow V_1(1) = 7$ at $x = 1$
- $s = 2$:
  - $x = 0$: $0 + 0.9 \cdot 12 = 10.8$
  - $x = 1$: $7 + 0.9 \cdot 7 = 13.3$
  - $x = 2$: $12 + 0 = 12$
    $\Rightarrow V_1(2) = 13.3$ at $x = 1$
- $s = 3$:
  - $x = 0$: $0 + 0.9 \cdot 15 = 13.5$
  - $x = 1$: $7 + 0.9 \cdot 12 = 17.8$
  - $x = 2$: $12 + 0.9 \cdot 7 = 18.3$
  - $x = 3$: $15 + 0 = 15$
    $\Rightarrow V_1(3) = 18.3$ at $x = 2$
- $s = 4$:
  - $x = 0$: $0 + 0.9 \cdot 16 = 14.4$
  - $x = 1$: $7 + 0.9 \cdot 15 = 20.5$
  - $x = 2$: $12 + 0.9 \cdot 12 = 22.8$
  - $x = 3$: $15 + 0.9 \cdot 7 = 21.3$
  - $x = 4$: $16 + 0 = 16$
    $\Rightarrow V_1(4) = 22.8$ at $x = 2$

Summary at $t = 1$:
$V_1(0) = 0,\ V_1(1) = 7,\ V_1(2) = 13.3,\ V_1(3) = 18.3,\ V_1(4) = 22.8.$

2.2.5 Step 3: Initial period $t = 0$ (with $s_0 = 4$)

Compute $V_0(4) = \max_{x \leq 4}\{\pi(x) + \beta V_1(4 - x)\}$:

- $x = 0$: $0 + 0.9 \cdot 22.8 = 20.52$
- $x = 1$: $7 + 0.9 \cdot 18.3 = 23.47$
- $x = 2$: $12 + 0.9 \cdot 13.3 = 23.97$
- $x = 3$: $15 + 0.9 \cdot 7 = 21.3$
- $x = 4$: $16 + 0 = 16$

$\Rightarrow$ **Optimal** $x_0^*(4) = 2$ and $V_0(4) = 23.97$.

2.2.6 Optimal path and value

Starting at $s_0 = 4$:

- $t = 0$: extract $x_0 = 2 \Rightarrow s_1 = 2$
- $t = 1$: with $s_1 = 2$, optimal $x_1 = 1 \Rightarrow s_2 = 1$
- $t = 2$: with $s_2 = 1$, extract all $x_2 = 1 \Rightarrow s_3 = 0$

Discounted payoff:

- $t = 0$: $\pi(2) = 12$
- $t = 1$: $\beta\pi(1) = 0.9 \cdot 7 = 6.3$
- $t = 2$: $\beta^2\pi(1) = 0.81 \cdot 7 = 5.67$

Total $= 12 + 6.3 + 5.67 = 23.97 = V_0(4)$. Thus, the optimal extraction policy is to extract 2 units at $t = 0$, 1 unit at $t = 1$, and 1 unit at $t = 2$, yielding a total discounted profit of 23.97.

See excel example for full table.

2.3 The Curse of Dimensionality

Further reading: M&F §7.3.4, pp. 166–167, relates computational work to the numbers of states, actions, and iterations.

Dynamic programming offers an elegant recursive solution principle, but it suffers from a major practical limitation known as the **curse of dimensionality**.

The term was coined by Richard Bellman to describe the exponential growth in computational complexity as the number of state or control variables increases.

2.3.1 Illustration

In our mine management problem, the state s_t took on a small number of discrete values (e.g., $s_t \in \{0, 1, 2, 3, 4\}$).
If we introduce additional state variables — such as capital, technology, or another resource — the number of possible state combinations grows rapidly.

For example:

Dimension	Grid points per variable	Total grid points
1 state variable	100	100
2 state variables	100	10^4

Dimension	Grid points per variable	Total grid points
3 state variables	100	10^6
4 state variables	100	10^8

Even modest discretizations quickly lead to millions of points. At each grid point, we must compute and compare values for all possible controls, making the full state–action space enormous.

Each additional state dimension multiplies the size of:

- The **value function table** $V_t(s)$
- The **policy function** $x_t^*(s)$
- The **transition mapping** $s_{t+1} = f(s_t, x_t)$
- And the computational effort of evaluating $\pi(s_t, x_t) + \beta V_{t+1}(f(s_t, x_t))$

This explosion makes naive enumeration infeasible.

2.4 Forest Harvest Problem

Further reading: M&F Exercise 7.1, p. 185, offers a related timber-harvest problem with different growth, rewards, and replanting timing. For the analogous keep-or-replace structure, see §7.2.2, pp. 158–159; for terminal and salvage values, see §7.1, p. 156.

We consider a simple **finite-horizon dynamic programming** problem for forest management. At each decision date, the forest is characterized by its *age*, and the decision is whether to **cut** or **wait**.

2.4.1 Model Elements

- **State variable**: s_t = forest age at the beginning of period t
- **Decision variable**: $x_t \in \{0, 1\}$
  - $x_t = 0$ means *wait*
  - $x_t = 1$ means *harvest*
- **Immediate payoff** $\pi(s_t, x_t)$ depends on both stand age and the harvest decision. Before the terminal date:
  - If we wait, $\pi(s_t, 0) = 0$.
  - If we harvest, the payoff depends on age: $\pi(0, 1) = -2$, $\pi(50, 1) = 8$, and $\pi(100, 1) = 10$.

$$\pi(s_t, x_t) = \begin{cases} 0, & x_t = 0 \quad \text{(wait)}, \\ -2, & x_t = 1,\ s_t = 0, \\ 8, & x_t = 1,\ s_t = 50, \\ 10, & x_t = 1,\ s_t = 100. \end{cases}$$

- **Transition**: $s_{t+1} = f(s_t, x_t)$, where

$$f(s_t, x_t) = \begin{cases} \min(s_t + 50, 100), & x_t = 0 \quad \text{(wait)}, \\ 0, & x_t = 1 \quad \text{(harvest)}. \end{cases}$$

- **Terminal value**: $V_T(s_T) = \max\{\text{salv}(s_T), \pi(s_T, 1)\}$ where
  - $\text{salv}(s_T)$ = salvage value if not harvested at end, $\text{salv}(0) = 0$, $\text{salv}(50) = 20$, $\text{salv}(100) = 60$
  - $\pi(s_T, 1)$ is the harvest payoff defined above. At the terminal date, leaving the stand uncut earns its salvage value instead of the zero immediate payoff from waiting in earlier periods.
- **Discount factor**: $\beta = (1/1.04)^{50} \approx 0.1405$ (discounting 50 years at 4%)

2.4.2 Bellman Equation

The Bellman equation for this problem is:

$$V_t(s_t) = \max_{x_t \in \{0,1\}} \left[\pi(s_t, x_t) + \beta \, V_{t+1}(s_{t+1}) \right],$$

where

- $\pi(s_t, x_t)$ is the **immediate payoff**, and
- $s_{t+1} = f(s_t, x_t)$ follows the **transition function** defined above.

We can solve this problem by **backward induction** starting from the terminal period T and moving backward to $t = 0$ without defining initial conditions.

2.4.3 Intuition

- What is the intuition behind the optimal policy? At each age s_t, we compare the value of harvesting now versus waiting for future growth.
- Even though the payoff function is discrete, we can still solve via backward induction by simulating outcomes at each state.

2.4.4 Implementation in R

Go to the R script `forest_harvest.R`

3 Infinite Horizon Problems

Time remains discrete throughout this section. We first study the fishery with continuous stock and harvest to derive the economic tradeoffs, then approximate it with discrete states and actions for numerical solution.

- In many economic applications, we consider **infinite-horizon** dynamic programming problems where the decision-maker optimizes over an infinite sequence of periods. With time-invariant model elements and suitable regularity conditions, we can use a **stationary policy** that depends on the current state but not on calendar time. A stationary policy need not produce a constant stock; a steady-state stock is a separate property of the resulting dynamics.
- The general infinite-horizon problem is to choose a sequence $\{x_t\}_{t=0}^{\infty}$ to maximize the expected discounted sum of single-period payoffs:

  $$\max_{x_t} \sum_{t=0}^{\infty} \beta^t \pi(s_t, x_t)$$

  subject to the deterministic transition dynamics:

  $$s_{t+1} = f(s_t, x_t),$$

  with an initial state s_0 given.
- The Bellman equation for an infinite-horizon problem is:

  $$V(s) = \max_{x \in X(s)} \{\pi(s, x) + \beta V(f(s, x))\}$$

  where we note that there is no time index on $V(s)$ since the value function is stationary (time autonomous).
- The key difference from finite-horizon problems is the absence of a terminal condition. Instead, we look for a fixed point of the Bellman operator. We can find a solution using **value function iteration** or **policy iteration** methods. Today, we focus on value function iteration.

3.1 Fishery Problem

Further reading: M&F §8.1, pp. 190–191, gives the continuous-state formulation; §8.4.2, pp. 198–200, develops renewable-resource harvesting. That example uses net social surplus with a demand curve and harvest costs; we use constant net revenue per unit and logistic growth. In §8.4.2, the book's harvest x is our h, and its growth function h is our G.

We study a fishery in **discrete time with continuous stock and harvest**. At each date, the manager observes the stock, chooses harvest, and leaves the remaining fish to grow before the next date.

3.1.1 Model Elements

Notation: Because this infinite-horizon model has payoffs, transitions, and feasible choices that do not depend on calendar time, we drop time subscripts below: s denotes current stock s_t, h denotes current harvest h_t, and $e = s - h$ denotes current escapement e_t. We write s' for next period's stock s_{t+1}. The prime on s means "next period," whereas primes on functions, such as V' and G', denote derivatives. The stationary value function is $V(s)$ rather than $V_t(s)$; the stock and harvest can still change over time.

- **State**: biomass $s \in [0, S_{\max}]$, where $S_{\max}$ is the carrying capacity.
- **Control**: harvest $h \in [0, s]$.
- **Reward/payoff**: $\pi(s, h) = (p - c)h$, with price $p > c \geq 0$ and constant cost per unit c.

- **Transition**: $s' = f(s,h) = G(s-h)$, where

$$G(e) = e + re\left(1 - \frac{e}{S_{\max}}\right).$$

  Here $e = s - h$ is **escapement**, the stock left after harvest, and r is the intrinsic growth rate. Assume $0 < r \leq 1$, so this growth function keeps next period's stock in $[0, S_{\max}]$.

- **Discount factor**: $0 < \beta < 1$.

The stationary Bellman equation is

$$V(s) = \max_{0 \leq h \leq s} \left\{(p-c)h + \beta V\big(G(s-h)\big)\right\}.$$

For a given stock s, this is a constrained nonlinear optimization problem in one variable, h. The additional challenge is that we do not yet know the continuation value function V.

3.1.2 Building Intuition

Further reading: M&F §8.2, pp. 191–193, derives shadow prices, first-order conditions, and the adjustments needed for binding constraints. See §8.4.2, p. 199, for their resource-harvest interpretation.

Suppose for now that we know V and it is differentiable at the relevant next-period stock. Define the objective at a given current stock as

$$Q_s(h) = (p-c)h + \beta V\big(G(s-h)\big).$$

Its derivative wrt h is

$$Q_s'(h) = p - c - \beta V'(s')G'(s-h), \qquad s' = G(s-h),$$

where

$$G'(s-h) = 1 + r\left(1 - \frac{2(s-h)}{S_{\max}}\right).$$

Here G' denotes the derivative with respect to escapement. Holding s fixed and differentiating with respect to harvest instead, the chain rule gives

$$\begin{aligned}\frac{\partial G(s-h)}{\partial h} &= \frac{\partial}{\partial h}\left[(1+r)(s-h) - \frac{r}{S_{\max}}(s-h)^2\right] \\ &= -(1+r) + \frac{2r(s-h)}{S_{\max}} \\ &= -G'(s-h).\end{aligned}$$

The minus sign comes from $\partial(s-h)/\partial h = -1$ and explains the negative continuation-value term in $Q_s'(h)$.

- **Harvesting one more unit** earns $p - c$ today.
- **Leaving that unit in the water** adds $G'(s-h)$ units to next period's stock, each worth $V'(s')$ next period. Discount that value by β.
- At an **interior optimum**, $0 < h^*(s) < s$, these marginal values are equal:

$$p - c = \beta V'(s')G'(s - h^*(s)).$$

The bounds matter too. For $s > 0$, the necessary first-order conditions can be written as

$$\begin{cases} Q'_s(0) \le 0, & h^*(s) = 0, \\ Q'_s(h^*(s)) = 0, & 0 < h^*(s) < s, \\ Q'_s(s) \ge 0, & h^*(s) = s. \end{cases}$$

These are the implications of the KKT conditions for this bounded choice. If the objective is concave, they also suffice for a global optimum; otherwise, we must compare candidate solutions. At $s = 0$, the only feasible choice is $h = 0$.

Further reading: M&F §8.2, p. 192, derives the action first-order condition and the envelope condition together, including state-dependent bounds.

Why we need the envelope condition. The harvest first-order condition tells us how to choose harvest given the marginal value of future stock, $V'(s')$. But what determines the marginal value of stock today, $V'(s)$? The envelope condition links these two marginal values. It lets us interpret the shadow value of fish without first solving for how optimal harvest changes with stock.

To see where it comes from, write the Bellman equation evaluated at the optimal harvest rule:

$$V(s) = (p - c)h^*(s) + \beta V(G(s - h^*(s))).$$

Suppose the optimum is interior and both V and h^* are differentiable locally. A change in stock affects value in two ways: directly, by leaving more fish available to grow, and indirectly, by changing the optimal harvest. Differentiating both sides with respect to s, with $s' = G(s - h^*(s))$, gives

$$\begin{aligned} V'(s) &= (p - c)\frac{dh^*(s)}{ds} + \beta V'(s')G'(s - h^*(s)) \left(1 - \frac{dh^*(s)}{ds}\right) \\ &= \beta V'(s')G'(s - h^*(s)) \\ &\quad + \underbrace{[(p - c) - \beta V'(s')G'(s - h^*(s))]}_{=0 \text{ by the interior harvest FOC}} \frac{dh^*(s)}{ds}. \end{aligned}$$

The coefficient on the policy response $dh^*(s)/ds$ is zero because harvest is already chosen optimally. A small adjustment in harvest therefore has no first-order effect on value at an interior optimum. Only the direct effect of extra stock remains:

$$\boxed{V'(s) = \beta V'(s')G'(s - h^*(s)).}$$

This is the **envelope condition**: to find the marginal value of the state, differentiate the Bellman objective with respect to the state while holding the optimal action fixed. Here the immediate payoff has no direct dependence on stock, so the direct effect comes entirely through future stock.

Combining the envelope condition with the interior harvest first-order condition gives

$$V'(s) = p - c.$$

An additional unit of stock is worth the net revenue from harvesting another unit when harvest is interior. This derivation applies locally at an interior solution; at a binding harvest bound, the constraint terms must also be considered.

3.1.3 Numerical Solution via Value Function Iteration

Further reading: M&F §7.3.2, p. 165, calls this algorithm "function iteration"; §7.5, pp. 169–172, shows its implementation in MATLAB/CompEcon. Our R script implements the discrete Bellman updates directly. These references explain the discrete algorithm, not a continuous-state numerical solver.

The theoretical model has continuous stock and harvest. For the numerical example, we approximate it with **discrete stock levels and harvest choices**. The economic tradeoff is the same, but we find the best harvest by comparing a finite list of feasible actions. The derivatives above explain the continuous model; we do not apply those first-order or envelope conditions to the discrete value table or rounded transition. Enumeration solves each numerical subproblem.

- Carrying capacity $S_{\max} = 20$ and growth rate $r = 0.5$.
- Discount factor $\beta = 0.95$, price $p = 1$, and unit cost $c = 0$.
- State grid $\mathcal{S} = \{0, 1, \ldots, 20\}$.
- At stock s, feasible harvests are $\mathcal{H}(s) = \{0, 1, \ldots, s\}$.
- Compute growth using $G(s-h)$, then round to the nearest grid point. Write this numerical transition as $\hat{f}(s,h) = \text{round}(G(s-h))$.

For example, leaving 7 units gives $G(7) = 7 + 0.5(7)(1 - 7/20) = 9.275$. The numerical model therefore moves to stock 9. This rounding is part of our approximation, not part of the biological model.

We write $\widehat{V}$ for a value table on the grid, distinguishing it from the continuous model's value function V. In the R script, this table is stored as `V`.

Solving the Bellman subproblems

Start with $\widehat{V}^{(0)}(s) = 0$ for every $s \in \mathcal{S}$. Using the previous iteration's values, update each stock level:

$$\widehat{V}^{(k+1)}(s) = \max_{h \in \mathcal{H}(s)} \left\{ (p-c)h + \beta \widehat{V}^{(k)}(\hat{f}(s,h)) \right\}.$$

At each stock level:

1. List every feasible harvest $h \in \mathcal{H}(s)$.
2. Compute its current payoff and rounded next-period stock.
3. Look up that next stock's value in $\widehat{V}^{(k)}$ and discount it by β.
4. Choose the harvest with the highest current payoff plus discounted continuation value. Store both the maximizing harvest and the maximized value.

Use the same previous-iteration value vector for every stock in an update. The outer VFI loop updates continuation values; the inner comparison chooses the best harvest given those values. At zero stock, the only feasible harvest is zero, and stock remains zero.

Repeat until

$$\max_{s \in \mathcal{S}} \left| \widehat{V}^{(k+1)}(s) - \widehat{V}^{(k)}(s) \right| < 10^{-8}.$$

Then compare the harvest choices once more using the final values to recover the policy at every stock on the grid. If several actions tie, any maximizing action is optimal for the discrete problem; use a consistent rule to select one.

Why value function iteration solves the problem

- **We want a decision rule for every state on the grid.** For each stock level, what harvest gives the highest current payoff plus discounted future value?
- **The challenge is valuing what we leave for tomorrow.** We know today's payoff and how the stock changes, but initially do not know the value of arriving at each future stock.
- **Start with a guess for future values.** With $\widehat{V}^{(0)} = 0$, the first update values only current harvest. Since $p > c$, it chooses $h = s$.
- **Use the updated values to compare harvest choices again.** Future stock now has value, so preserving fish can become worthwhile. Use the same previous-iteration values for every state in an update.
- **Each update incorporates another period of consequences.** Starting from zero, the first update solves a one-period problem, the second a two-period problem, and so on, for our discrete model. The index k counts iterations, not calendar periods.
- **Discounting makes distant payoffs matter less.** With bounded payoffs and $\beta < 1$, extending the planning horizon eventually adds very little to the value at any state.
- **Stop when the values barely change across the grid.** Optimizing with those continuation values then approximately reproduces the values we started with.
- **Recover the policy from the final values.** An initial stock is needed only afterward, to trace the path generated by following that policy.

Connecting back to the continuous model. With the same parameters and sufficiently fine grids, we expect the discrete solution to approximate the continuous model's values and decisions. A coarse grid can change harvest thresholds and create apparent steady states because of rounding. To improve the approximation, reduce the spacing of both stocks and harvests, for example from 1 to 0.5 to 0.25, and round growth to the nearest point on the chosen state grid. Compare values and policies at common stock levels.

Further reading: M&F §7.3.2, p. 165, gives a bound on iteration error for the discrete model. That bound concerns convergence to the discrete solution, not the additional error from discretizing a continuous model.

Convergence versus accuracy. A small change between iterations tells us that we have nearly solved the discrete model. It does not establish that this model closely approximates the continuous problem. Grid refinement checks that separate source of error; tightening the VFI stopping tolerance alone does not remove it.

Further reading: M&F §7.4, pp. 167–168, explains how an optimal policy generates state paths and how to identify deterministic steady states. This is the basis for interpreting our stock-dynamics figure.

The R script `fishery.R` implements this enumeration-and-rounding approach. It uses the parameters and state grid listed above and produces plots of convergence, optimal harvest, stock dynamics, and harvest versus escapement. At exact half-integers, R rounds to the nearest even integer; the script uses this rule consistently.

3.2 A bit more depth: The Bellman Operator

Further reading: M&F §7.3, pp. 163–165, writes the discrete Bellman update as a vector fixed-point problem. Compare §8.1, p. 191, for the continuous-state functional fixed-point equation. We use $\mathcal{T}$ to name the updating rule.

The Bellman equation tells us that the value of being in a given state today, $V(s)$, equals the best possible current payoff plus the discounted value of what happens next.

We can think of this process as an **operator** — a kind of "machine" that takes a guess about the value function and produces a new, updated guess.

3.2.1 How it works

Start with any function $V(s)$ that tells you what the value might be for each state.
Then define a new function:

$$(\mathcal{T}V)(s) = \max_{x \in X(s)} \{\pi(s,x) + \beta V(f(s,x))\}.$$

Reading the notation:

- $\mathcal{T}$ is the **Bellman operator**, a rule that maps one function into another. It is not itself an equation.
- $\mathcal{T}V$ is the **updated function** obtained by applying that rule to the candidate value function V. This notation means applying an operator, not multiplying by $\mathcal{T}$.
- $(\mathcal{T}V)(s)$ is the **number** that the updated function assigns to a particular state s.

Value function iteration repeatedly applies this rule: $V^{(k+1)} = \mathcal{T}V^{(k)}$.

Continuous model versus discrete approximation. For the theoretical fishery, $X(s) = [0, s]$ (with action h) and $f(s,h) = G(s-h)$. For the numerical fishery, define a separate operator on value tables:

$$(\widehat{\mathcal{T}}W)(s) = \max_{h \in \mathcal{H}(s)} \left\{(p-c)h + \beta W(\hat{f}(s,h))\right\}, \qquad s \in \mathcal{S}.$$

The R script performs $\widehat{V}^{(k+1)} = \widehat{\mathcal{T}}\widehat{V}^{(k)}$. Both operators use the same economic logic, but they solve different problems: one has continuous choices and unrounded growth, and the other has grid choices and rounded growth.

What this says in words:

"Given my current guess about how valuable future states are (that's V), what would be the total value of making the best decision today?"

So $\mathcal{T}$ takes the *old* value function and gives you a *new* one that's a little closer to the truth.
It's a way to **think one step ahead**.

3.2.2 The Fixed Point Idea

The true value function, $V^*(s)$, is the one that **doesn't change** when we apply this operation again:

$$V^*(s) = (\mathcal{T}V^*)(s).$$

Requiring this equality at every state gives the **Bellman functional equation**, written compactly as $V^* = \mathcal{T}V^*$. Its unknown is the entire function V^*. The discrete model instead has its own fixed point $\widehat{V}^* = \widehat{\mathcal{T}}\widehat{V}^*$. This table approximates V^* at grid points; the two need not coincide even after numerical iteration has converged. Thus, $\mathcal{T}$ names the updating rule, while $V^* = \mathcal{T}V^*$ is the equation that the optimal value function must satisfy.

In other words, if you already know the correct V^*, thinking one step ahead doesn't change your beliefs because you are already correct about the future.

Why this matters

- The operator gives us a **recipe for computing** V^*:
  - start with a guess and keep applying $\mathcal{T}$ repeatedly.
  - Each time, you're improving your estimate of the value of being in each state.
- Economically, this process mirrors **learning or planning**:
  - we evaluate today's decisions using our expectations of tomorrow, adjust, and repeat until everything is internally consistent.

Visual Intuition

- If you plotted $V^{(0)}(s)$ (your first guess) and then $V^{(1)}(s) = \mathcal{T}V^{(0)}(s)$, the curves would move closer and closer together until they line up at $V^*(s)$.
- That's what it means for the Bellman equation to be a **fixed point** — a steady state in your expectations about value.

Each application of $\mathcal{T}$ corresponds to "thinking one step further ahead."

- Starting with any initial guess $V^{(0)}(s)$,
- Repeatedly applying $\mathcal{T}$, $V^{(k+1)} = \mathcal{T}V^{(k)}$,
- Converges to the true value function V^* under mild conditions.

This is **Value Function Iteration (VFI)**.

3.2.3 Contraction Mapping: Why Value Function Iteration Converges

Further reading: M&F §7.1, p. 157, and §7.3.2, p. 165, discuss contraction, uniqueness, and convergence for discrete models; §8.1, p. 191, gives the continuous-state counterpart. For an alternative algorithm, see policy iteration in §7.3.3, pp. 165–166.

Once we define the Bellman operator $\mathcal{T}$ - the rule that takes a guess about future value and updates it - we can ask two key questions:

1. Does this process always lead to a stable value function?
2. Will it always settle on the **same** function, no matter where we start?

For the continuous fishery model, bounded continuous payoffs, continuous transitions, compact feasible choice sets, and $0 < \beta < 1$ give a well-defined Bellman operator on continuous value functions. The contraction argument below applies to this operator. It also applies separately to $\widehat{\mathcal{T}}$, since the discrete problem has bounded payoffs and nonempty finite action sets. Each operator converges to its own fixed point.

The reason is the **contraction mapping property**.

A **contraction mapping** is a transformation that *pulls things closer together* every time you apply it.

$$\|\mathcal{T}W - \mathcal{T}Z\|_\infty \leq \beta \|W - Z\|_\infty$$

Here, $\|W - Z\|_\infty$ is the largest absolute difference between the two value functions across all states. One Bellman update reduces this maximum difference by at least the factor β.

Imagine taking two different guesses about the value function, say $W(s)$ and $Z(s)$. When we apply the Bellman operator to both, the resulting functions $\mathcal{T}W$ and $\mathcal{T}Z$ are **closer to each other** than the originals.

Each round of updating reduces the distance between our guesses — eventually, all sequences converge to the same point, the **true value function** V^*.

Discounting is what makes this work. Because future rewards are multiplied by $\beta < 1$, any disagreement about the future is automatically **shrunk** when we think one step ahead.

Small differences in how we value the future can't explode backward into large differences today — they fade over time.

This gives the Bellman operator its *gravitational pull* toward a single stable value function.

If applying $\mathcal{T}$ repeatedly always pulls guesses closer together, there must be one and only one function that can't be improved upon — that's the **fixed point**:

$$V^*(s) = \mathcal{T}V^*(s).$$

Mathematically, this follows from the **Contraction Mapping Theorem**, but economically, you can think of it as saying:

There is one internally consistent way to value the future that agrees with itself when we plan forward.

3.2.3.1 How this helps us in practice

Because $\mathcal{T}$ is a contraction, we can find V^* by **value function iteration**:

1. Start with any initial guess $V^{(0)}(s)$ — even something crude.
2. Apply $\mathcal{T}$ to get an updated guess $V^{(1)} = \mathcal{T}V^{(0)}$.
3. Keep repeating: $V^{(k+1)} = \mathcal{T}V^{(k)}$.

Each iteration gets us closer to the truth.
Each update still requires solving the maximization problem at every state. Value function iteration organizes these optimizations into successive improvements in our estimate of future value. With a numerical grid, we converge to an approximate solution whose accuracy also depends on state and action grid spacing and the approximation of transitions.

3.2.3.2 The big takeaway

- On our finite grids, bounded payoffs, nonempty feasible action sets, and $0 < \beta < 1$ guarantee a **unique discrete value table** $\widehat{V}^*$ and convergence of value function iteration to that table. This guarantee does not measure its distance from the continuous solution V^*.
- Diminishing returns are not required for this result.
- A unique value function does not require a unique optimal action: two actions can tie for the same highest value.

4 Functional Equations and Value Function Properties

Further reading: M&F Chapter 8 introduction and §8.1, pp. 189–191, explain why continuous-state dynamic models lead to functional fixed-point problems.

The Bellman functional-equation idea applies to both continuous and discrete state models. The derivative and envelope arguments below concern continuous states and controls with the stated smoothness and interiority assumptions; they are not derivatives of the discrete fishery value table.

Dynamic programming problems often lead to **functional equations**—equations in which the *unknown* is a function rather than a scalar. The **Bellman equation** is the most important example in economics. Instead of solving for a scalar like x, we are solving for an entire function $V(\cdot)$ that satisfies a relationship involving itself.

4.1 What Is a Functional Equation?

- A **functional equation** relates the value of a function at one point to its value at another point.
- In its simplest form, it can look like:

$$V(s) = F(s, V(g(s)))$$

- where the function V appears on both sides.

- We are not trying to find a single number, but rather a function $V(\cdot)$ that makes this equation true for every s.
- Functional equations appear whenever we describe **recursive behavior** — situations where "the value of something today depends on the value of something tomorrow."

Think of **compound interest**:

$$A_{t+1} = (1 + r)A_t$$

The future value depends on today's value through a simple rule.

A functional equation specifies a relationship between a function and its transformed version.

In dynamic optimization:

$$V(s) = \max_{x \in X(s)} \{\pi(s, x) + \beta V(f(s, x))\}$$

where:

- $V(s)$ is the **value function**, giving the maximum attainable value from state s.
- $\pi(s, x)$ is the **current payoff** (or profit, utility).
- $f(s, x)$ gives the **next-period state**.
- $\beta \in (0, 1)$ is the **discount factor**.

The equation says: *The value of being in state s today equals the current payoff plus the discounted value of the next state, assuming optimal choice x.*

4.1.1 Functional Equations in Economics

Example: A Simple Savings Problem

Further reading: M&F §8.4.1, pp. 197–198, studies a related consumption–investment problem and interprets its Euler equation. Its production technology differs from our fixed-interest-rate savings example.

Suppose an agent chooses consumption c_t and next-period assets a_{t+1}.

$$\max_{\{c_t, a_{t+1}\}_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t \pi(c_t)$$
$$\text{s.t. } c_t + a_{t+1} = (1+r)a_t + y, \quad a_t \geq 0$$

where:

- $\pi(c)$ is the single-period payoff, interpreted here as utility (e.g., $\pi(c) = \log(c)$),
- r is the interest rate,
- y is constant income in every period (we assume it does not vary with time),
- $\beta \in (0, 1)$ is the discount factor.

Then the Bellman equation is

$$V(a_t) = \max_{a_{t+1} \geq 0} \{\pi((1+r)a_t + y - a_{t+1}) + \beta V(a_{t+1})\}.$$

FOC (interior) with respect to a_{t+1}

Let $c_t = (1+r)a_t + y - a_{t+1}$. Then

$$\frac{\partial}{\partial a_{t+1}} \Big[\pi((1+r)a_t + y - a_{t+1}) + \beta V(a_{t+1})\Big] = -\pi'(c_t) + \beta V'(a_{t+1}) = 0,$$

Here we retain π for the payoff function even though the payoff is utility rather than profit. The agent trades off **current utility** $\pi(c)$ versus the **future value** $\beta V(a')$.

Firm Investment Problem

A firm with capital stock k_t chooses investment i_t:

$$\max_{i_t} \sum_{t=0}^{\infty} \beta^t \pi(k_t, i_t) \quad \text{s.t.} \quad k_{t+1} = (1-\delta)k_t + i_t, \quad k_t \geq 0.$$

The recursive form:

$$V(k_t) = \max_{i_t \geq 0} \{\pi(k_t, i_t) + \beta V((1-\delta)k_t + i_t)\}.$$

The function $V(k_t)$ gives the value of capital k_t as current profit plus discounted future (optimal) value of next-period capital. Again, the value function appears on both sides.

FOC with respect to i_t:

$$\frac{\partial}{\partial i_t} \Big[\pi(k_t, i_t) + \beta V((1-\delta)k_t + i_t)\Big] = \pi_i(k_t, i_t) + \beta V'((1-\delta)k_t + i_t) = 0.$$

Resource Extraction Problem

Further reading: M&F §8.4.3, pp. 200–201, extends extraction to stock-dependent costs, a price that depends on extraction, and possible abandonment.

A resource owner decides how much to extract x_t from stock s_t:

$$V(s_t) = \max_{0 \leq x_t \leq s_t} \{px_t - c(x_t) + \beta V(s_t - x_t)\}.$$

Here:

- The **state** is remaining stock s_t.
- The **control** is extraction x_t.
- The **transition** is $s_{t+1} = s_t - x_t$.

The functional equation states that the value of the resource today equals profit from extraction plus the discounted value of what remains for tomorrow.

4.2 The Envelope Theorem in Dynamic Programming

Further reading: M&F §8.2, pp. 191–193, develops the shadow-price and Euler conditions. Page 192 shows how the envelope condition changes when action bounds depend on the state.

Consider a general Bellman equation:

$$V(s) = \max_{x \in X(s)} \{\pi(s,x) + \beta V(f(s,x))\}.$$

Let $x^*(s)$ be the optimal policy function.
We often need the derivative $V'(s)$ to derive first-order conditions or Euler equations.

4.2.1 The Problem

- The right-hand side depends on s **both directly** (through π and f) and **indirectly** through the optimal choice $x^*(s)$.
- If we differentiate $V(s)$ directly, we would in principle need to differentiate through $x^*(s)$, which is messy.
- The envelope theorem tells us we can ignore that part.

4.2.2 The Envelope Condition

At a differentiable interior optimum $x^*(s)$, with no binding state-dependent constraints,

$$V'(s) = \left.\frac{\partial \pi(s,x)}{\partial s}\right|_{x=x^*(s)} + \beta\, V'(f(s,x^*(s)))\, \left.\frac{\partial f(s,x)}{\partial s}\right|_{x=x^*(s)}.$$

Intuition:

- Because the first-order condition for x^* holds, the indirect effect of changing s through a change in $x^*(s)$ is *zero*.
- We only need to consider the **direct** effects of s on the current payoff and next-period state.

4.2.3 Dynamic Envelope Condition (Benveniste–Scheinkman Theorem)

In discrete time, the theorem states:

- If the Bellman equation is differentiable and the policy function $x^*(s)$ is interior and continuous, then

$$V'(s) = \frac{\partial}{\partial s}\Big[\pi(s,x) + \beta V(f(s,x))\Big]\Big|_{x=x^*(s)}.$$

- You can interpret $V'(s)$ as the **shadow value of the state variable** - the marginal increase in lifetime value from a marginal increase in the current state.

4.2.4 Example: Consumption–Savings Problem

Bellman equation:

$$V(a_t) = \max_{a_{t+1}} \{\pi((1+r)a_t + y - a_{t+1}) + \beta V(a_{t+1})\}.$$

FOC w.r.t. a_{t+1}:

$$-\pi'(c_t) + \beta V'(a_{t+1}) = 0 \quad \rightarrow \quad \pi'(c_t) = \beta V'(a_{t+1}).$$

Envelope condition:

$$V'(a_t) = \pi'(c_t) \cdot (1+r).$$

Differentiate the Bellman RHS w.r.t. a_t holding the optimizer fixed:

$$V'(a_t) = \pi'(c_t) \cdot (1+r).$$

Euler equation

Apply the envelope condition at $t+1$:

$$V'(a_{t+1}) = (1+r)\pi'(c_{t+1}).$$

Substitute into the FOC:

$$\pi'(c_t) = \beta(1+r)\pi'(c_{t+1}).$$

Interpretation:

- An extra dollar of wealth today raises lifetime value by $(1+r)\pi'(c_t)$ - it can either be consumed now (worth $\pi'(c_t)$) or saved to yield $(1+r)\pi'(c_{t+1})$ next period. The household is indifferent between consuming today and saving for tomorrow when the Euler equation holds.

4.2.5 Why This Matters

- The envelope theorem allows us to link **marginal value** ($V'(s)$) to **marginal utility** or **marginal profit**, depending on context.
- It simplifies derivations of **Euler equations**, **costate equations**, and **first-order necessary conditions**.
- It provides a recursive way to compute derivatives of value functions in both analytical and numerical work.